DaVinci Lens - Project Report

By Beatrice Marro

Fontys University of Applied Sciences
S7 Enhanced AI Techniques

Contents

Abstract 2
Introduction 2
Background and Motivation 2
Problem Statement 3
Domain Study 3
Visual Differences Between Painting Mediums 3
Data 4
Sources and Acquisition 4
Initial Dataset Issues 4
Supplementation of Minority Classes 5
Full Image vs Tile-Based Extraction 6
Data Augmentation Strategy 6
Tile-Based Workflow 7
Auto-Crop Frame 7
Split into Tiles 8
Filtering Noisy Tiles 8
Prediction and Majority Vote 10
Modelling 11
Iterative Development Overview 11
Iteration 1 – Support Vector Machine (SVM) 11
Iteration 2 – Various VGG16 Models 12
Iteration 3: Final Model Configuration (EfficientNetV2-S) 13
Optimal Tile Size 14
Evaluation 15
Tile-level Performance 15
Full-Artwork Performance 15
Confusion Patterns 16
Error Analysis and Model Interpretability (Grad-CAM) 17
Discussion 17
Key Findings and Lessons Learned 17
Limitations 17
Conclusion and Presentation 18
Summary of Achievements 18
Web Application 18
References 21

Abstract

Identifying the medium of a painting (oil, tempera, watercolour, gouache and fresco/mural) is challenging among non-experts due to the subtle differences in texture, opacity, brushwork and pigment behaviour. The project developed is DaVinci Lens, which is an AI-powered tool that enables users to scan a painting to automatically classify its artistic medium.

The approach uses the ART500K toy dataset with additional paintings for gouache and watercolour from the Cleveland Museum of Art Open Access API. The dataset had a several class imbalances, fresco/mural noise in the pictures and frames being present on some paintings, this required extensive manual data cleaning. Once this was done, the end-to-end data cleaning and modelling pipeline had to be developed, this consists of automated anti-frame cropping, dark/blank tile filtering and tile-splitting. The artworks are split into 150x150 pixel tiles to focus on the local medium-specific clues.

Various modelling iterations were developed, starting with VGG16 (peak accuracy was ~76% but showing severe signs of overfitting), the final model uses a frozen EfficientNetV2-S backbone with a tuned classification head, achieving balanced ~75% per-tile accuracy without overfitting. At inference, majority voting across filtered tiles from the full artworks achieves ~93% accuracy (validated on 500 paintings).

The prototype demonstrates a strong performance on this difficult fine-grained visual task, a web application has developed for real-time scanning, this also includes information about the painting and art movement via a VLM as well as information about the colours and pigmentation though applied statistics. The limitations of this model include confusion between water-based media (watercolour and gouache) and a dataset bias towards historical Western art.

Introduction

This project addresses the challenge of automatic medium classification in paintings using deep learning. The goal is to develop a reliable model capable of distinguishing between five common historical mediums (oil, tempera, watercolour, gouache, and fresco/mural), based solely on local visual cues such as texture and pigment application.

Background and Motivation

Identifying the medium of a painting is something that art historians, conservators and auction houses do on a frequent basis, which is often difficult to achieve for non-experts. This difficulty is due to the presence of very subtle visual clues that require close inspection or sometimes even scientific analysis. Making this tool accessible gives great educational content for learning more about art for casual users, students and museum visitors among others.

Problem Statement

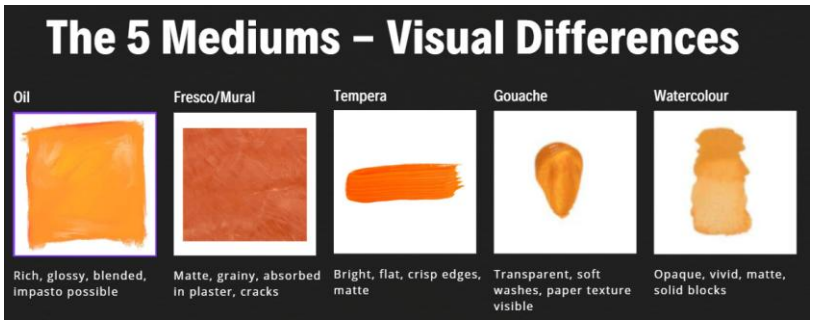
The core problem is the high level of difficultly required to effectively differentiate among mediums without extensive expertise in the art domain. The core task achieved by the DaVinci Lens project was to build an AI system that takes an image of a painting and predicts its medium.

Domain Study

Visual Differences Between Painting Mediums

Understanding the physical and visual differences of each medium is the logical and essential first step for human experts and the AI classification system alike to perform this task effectively.

Figure 1: *Painting Medium Visual Differences*



It is also important to understand the composition of these mediums and their unique properties in a greater depth.

Table 1: *Findings for differences in dataset mediums*

Medium	Color / Pigmentation	Opacity / Transparency	Brushstroke / Texture	Finish	Effect of Time / Aging	Typical Supports	Special Notes / Visual Cues
Oil	Rich, deep, highly saturated	Opaque; can be thin or thick	Can be smooth or heavily textured (impasto)	Glossy to semi-gloss	Darkens slightly, can yellow over centuries	Canvas, wood panels, metal	Subtle gradients, smooth blending; visible impasto possible; edges soft
Fresco / Mural	Muted, limited palette	Matte, pigments absorbed in plaster	Slightly grainy, integrated with wall	Matte	Colors generally stable but can fade or flake	Walls, ceilings, plaster	Often large scale; architectural context; rougher texture; uniform pigment absorption
Tempera	Bright, clean, flat	Mostly opaque	Fine linear strokes, cross-hatching	Matte	Colors remain stable; rarely darkens	Wood panels, parchment	Tiny hatch marks; no glossy sheen; flat color planes; crisp edges
Watercolour	Soft, pale, subtle	Transparent or semi-transparent	Smooth, flowing, washes	Matte	Paper may yellow; pigments can fade	Paper, vellum	Soft gradients; paper texture visible; layers show bleeding and transparency; often subtle edges
Gouache	Bright, vivid	Opaque (unlike watercolor)	Smooth, can layer over previous strokes	Matte	Colors stable; slight chalky effect over time	Paper, board, vellum	Solid color blocks; matte finish; less transparency; can cover mistakes or layers fully

These differences (especially in opacity, texture and brushwork), form the basis for the model to learn to distinguish between mediums successfully.

Data

Sources and Acquisition

For this project, the ART500K Dataset was used, as it contains a large collection of artworks from a wide variety of painters, techniques, and eras. This dataset, developed by the Hong Kong University of Science and Technology, is high-quality, complete, and fully labelled. This project uses the toy artwork dataset with 43,455 images.

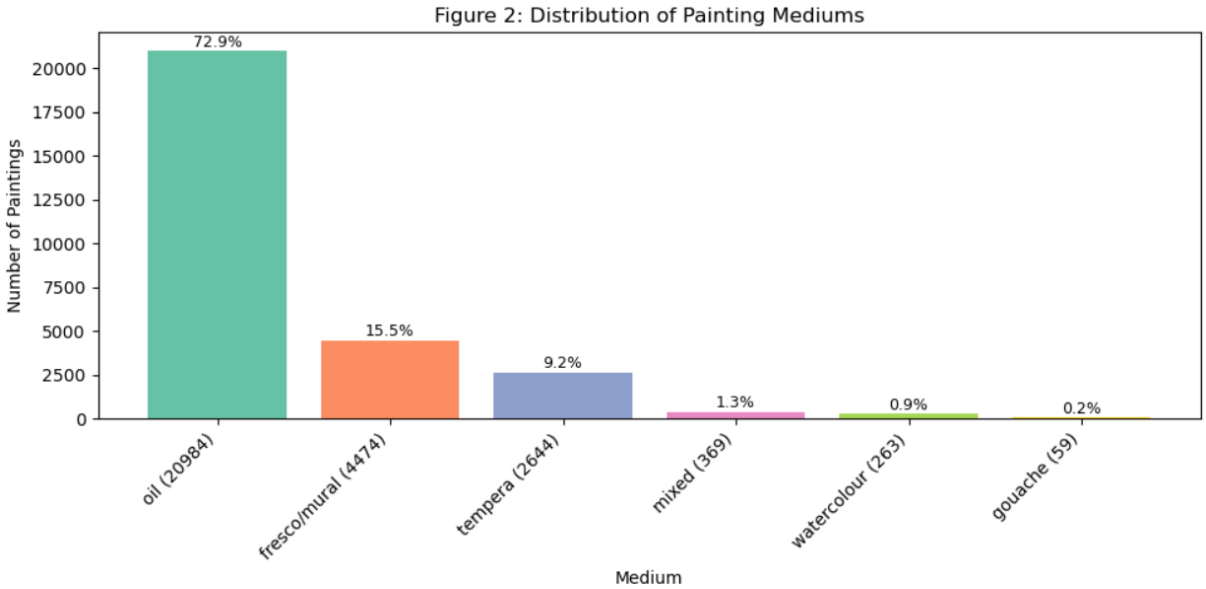
To compensate for the insufficient quantity of gouache and watercolour artworks, the Cleveland Museum of Art Open Access API has been used to download suitable images. This is the API of an art museum, containing 64,000 artwork records, where these are high-quality images making them suitable for model training.

Initial Dataset Issues

The initial (ART500K) dataset had multiple issues, these are listed below:

1. Severe class imbalance (oil ≈73%, gouache only 59 images, watercolour 263).

Figure 2: *Dataset Class Imbalances*



2. Noisy fresco/mural images that include full rooms (containing architecture, arches and furniture), which are irrelevant to the medium texture.

Figure 3: *Noisy Full Room Fresco/Mural Images*

3. Thick frames present around several paintings, which introduced the risk of the model learning the frame patterns instead of the paint characteristics.

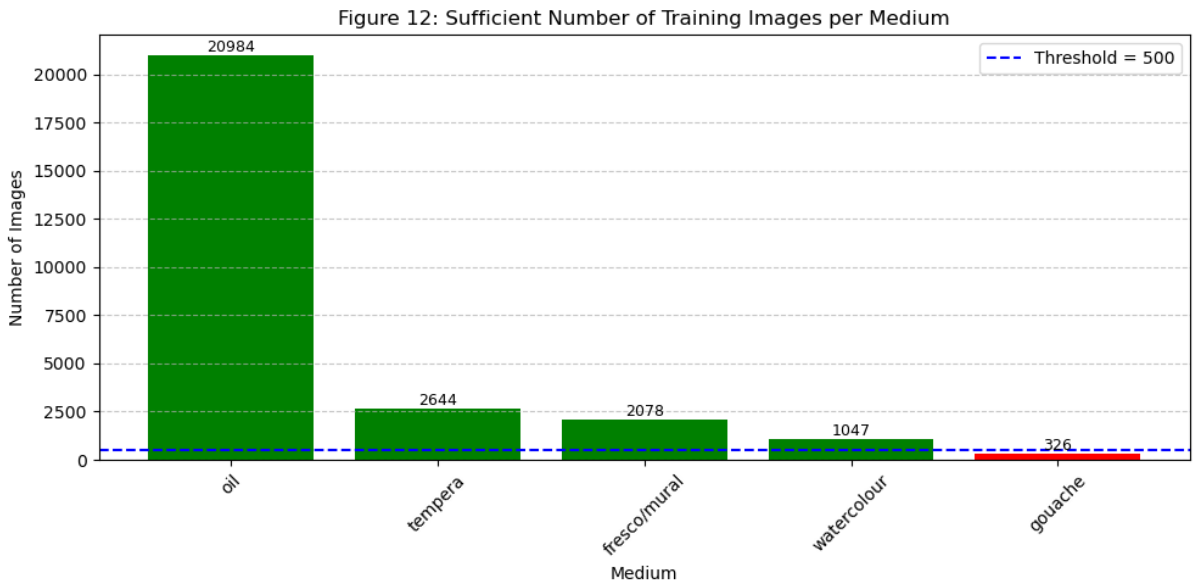
Figure 4: *Thick Frames Dataset Examples*

4. Other quality problems: dark/black tiles, plain paper in water-based media and inconsistent brightness (see section: *Tile-Based Workflow*).

Supplementation of Minority Classes

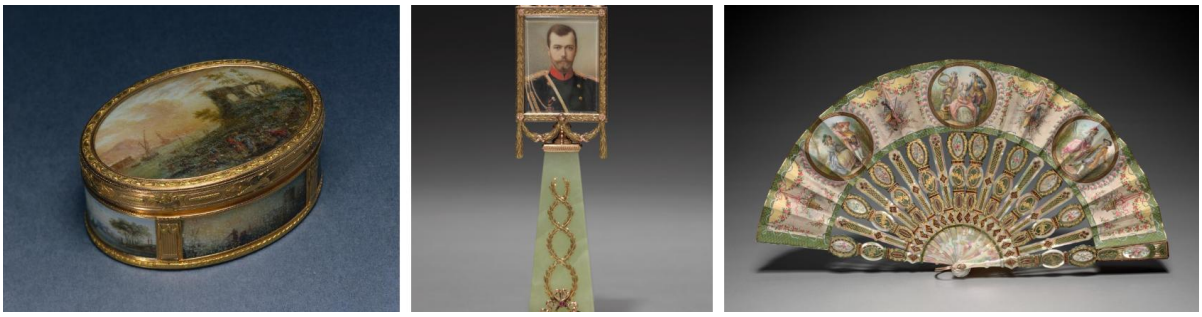
The water-based media (gouache and watercolour) had insufficient paintings to train a model, and the dataset was highly imbalanced (see Figure 2), which had 59 and 263 artworks respectively. This lack of data was compensated via adding paintings from an additional dataset, the Cleveland Open Access Museum API.

Figure 5: *Quantity of Images per Class after Additional Dataset Integration*



Once the additional gouache and watercolour artworks had been extracted, it was necessary to manually review these, as some artworks did not have a suitable (flat) support, and had to be removed manually.

Figure 6: *Unsuitable Removed Water-Based Medium Artworks*



This process resulted in bringing the minority classes closer to the viable training size when combined with the tiling.

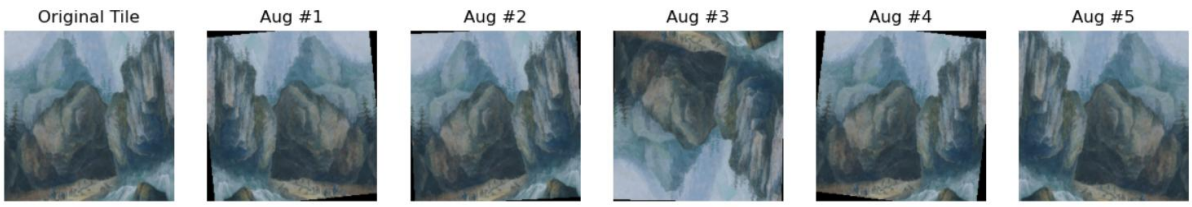
Full Image vs Tile-Based Extraction

As an experiment towards the best-performing strategy, the attempt to split the artworks into tiles or using the full image was investigated. The better resulting approach with improved performance was that of splitting paintings into 150x150 pixel tiles. This helps the model focus on local texture/brushwork as opposed to the subject matter or the global composition. Additionally, this increases the dataset size and allows for the opportunity to remove noisy tiles (see section: *Filtering Noisy Tiles*).

Data Augmentation Strategy

Given that the quantity of water-based media artworks is still significantly inferior to that of other mediums to reach a suitable level of training painting tiles for the model, a data augmentation strategy has been implemented to apply on-the-fly data augmentation during training. This data augmentation specifically does horizontal flipping, random rotation ±10° and random zooming ±10%.

Figure 7: *Data Augmentation for Sample Gouache Tile*



The purpose of this is to increase the dataset size, improving generalization and reducing overfitting (which is especially important for the minority classes).

Tile-Based Workflow

The goal of the tile-based workflow is to be able to turn a full painting into a reliable medium prediction.

Auto-Crop Frame

The problem behind the auto-cropping of the frame is that several paintings in the dataset include thick, ornate or colourful frames, where the risk lies in the model learning the frame patterns (such as gold edges which are often associated with tempera) instead of the actual painting medium.

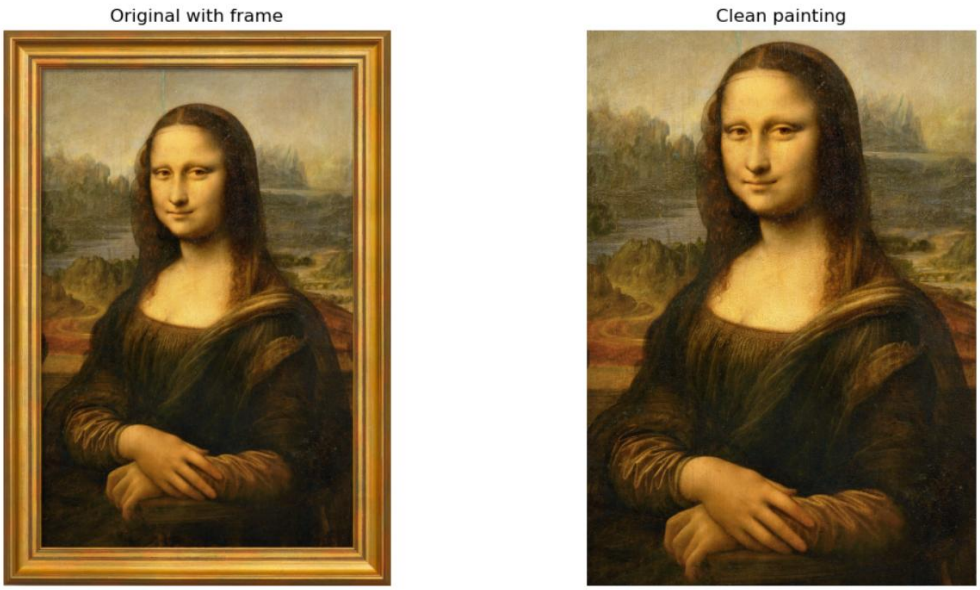
The solution implemented works by using a custom border-focused detection algorithm that only analyses the outer ~12% if the image to avoid touching the painting content.

The steps implemented to achieve this are the following:

1. High-threshold Canny edge detection on border region.
2. Hough line transforms to find long, straight, near-edge lines.
3. Requires at least 2 strong horizontal and 2 vertical lines close to the image borders.
4. Confidence check is performed to verify if the lines form a clear rectangle near the edges, where it should be cropped conservatively with a small padding.
5. If it is uncertain or there is no clear frame, it keeps the full image as a safe default mechanism.

The resulting algorithm removes the noisy frame tiles without accidentally cropping important parts of the painting.

Figure 8: *Auto-Frame Cropping Example*

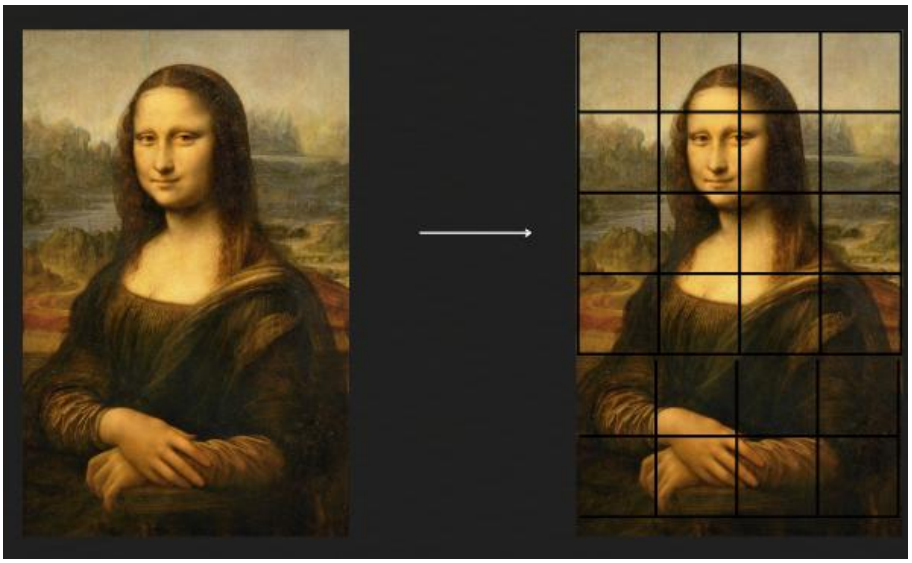


Split into Tiles

The artwork gets split into tiles instead of full images as to ensure that the model focuses on the local texture, brushstrokes and the pigment application. This is beneficial as it removes bias such as religious figures being dominant in tempera artworks. Additionally, it creates more data by splitting the painting up into tiles, which is especially useful for the minority classes. This approach also enables for the filtering of noisy tiles (see section: *filtering noisy tiles*).

This is achieved by splitting the artwork into unique 150x150 pixel tiles.

Figure 9: *Tile Splitting Example*



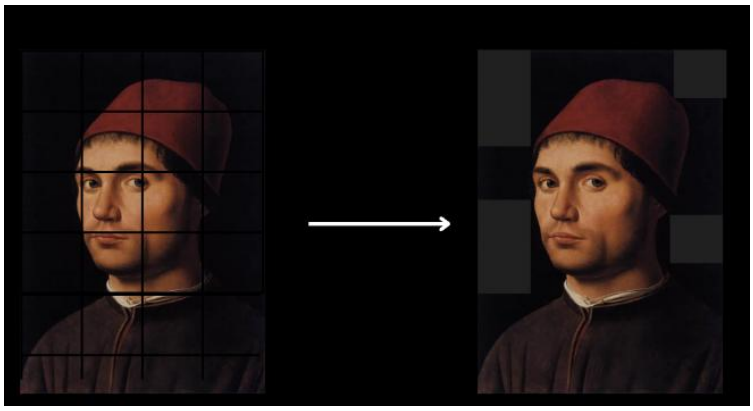
Filtering Noisy Tiles

Some of the tiles extracted from a painting do not containing any useful medium information (when they are too dark, completely blank or when plain paper or a wall is present).

The 3 main filters described below are applied after the tile splitting:

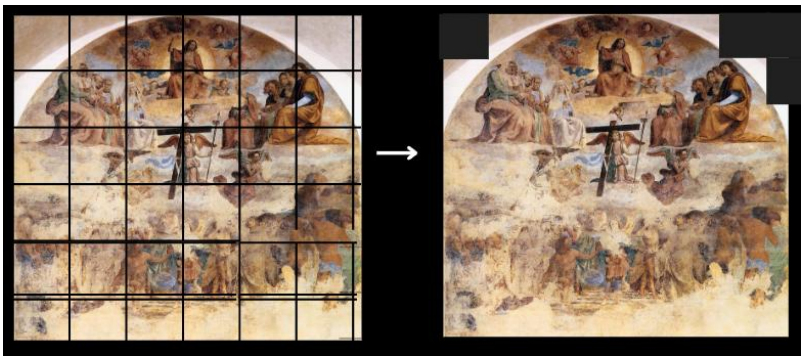
1. **Dark Tile Removal**
   - The tile is discarded when there is low mean greyscale brightness (< ~30–40) and a high proportion of near-black pixels.
   - This prevents the model from asocial darkness to a specific media (incorrectly).

Figure 10: *Black / Dark Tiles Removal Example*



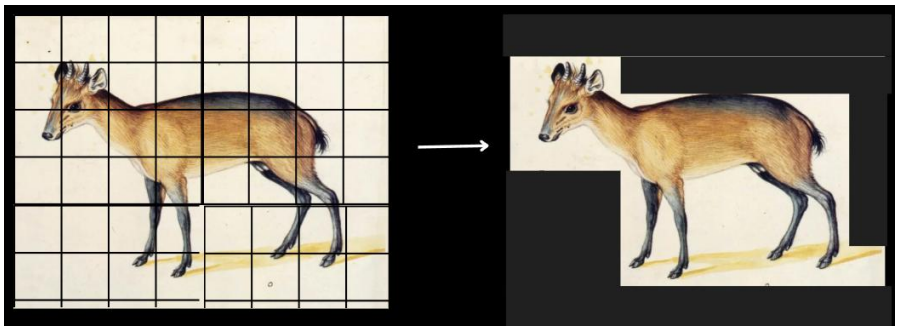
2. **Blank / Light Tile Removal**
   - The tile is discarded when there is a high proportion (>55-90%) of while/light grey/light-yellow pixels in HSV.
   - This targets blank paper (prominently present on water-based media), unpainted plaster (fresco/mural) and empty sky/wall areas.

Figure 11: *Complex Paper/Plain Wall/Blank Tiles Removal Example*



3. **Low-Variance Blank Paper / Wall Removal**
   - The tile is discarded when a high light ratio, but very low colour/intensity variance is present in non-light regions.
   - This catches small monochrome paper tiles or flat wall with almost no paint variation.

Figure 12: *Simple-Flat Paper/Plain Wall/Blank Tiles Removal Example*



The above-mentioned filters are applied sequentially; this enables the extraction for the valid tiles used in training and inference.

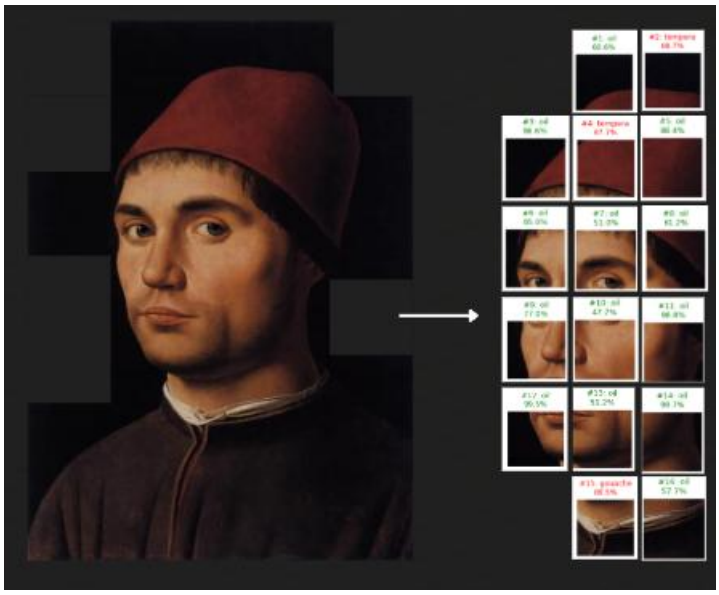
Prediction and Majority Vote

The prediction is made by the model following this end-to-end pipeline:

1. **Auto-crop frame** (if detected)
2. Split into **150x150 tiles**
3. Apply same **noisy-tile filters**, this keeps only the valid tiles
4. Run **EfficientNetV2-S** custom trained model on every valid tile, this outputs the class probabilities.
5. Takes the **argmax** per tile to collect the predicted labels.
6. The **final prediction** corresponds to the **majority (modal)** of all the tile predictions.

The majority vote is an effective measure of achieving this as the individual tiles can ambiguous or noisy, in which the errors are normally uncorrelated. Additionally, this approach aggregated many tiles (often ranging 10-100 per painting), this reduces the variance and increased the variability due to a greater spread.

Figure 13: *Tile-Based Workflow Prediction and Majority Vote Example*



Using this method, the **per-tile accuracy level** is **75%** and the **full painting accuracy** yielded **93%** (*see section: evaluation*).

Modelling

The model development followed an iterative approach, closely tied to the data which was constantly improved based on the error analysis.

Iterative Development Overview

Iteration 1 – Support Vector Machine (SVM)

The first iteration was done using without deep learning to prove the need for this, a traditional machine learning baseline was established using **Support Vector Machine (SVM)** to evaluate if using **hand-crafter feature filters** would solve this task efficiently. This feature engineering was achieved using **heavy combined filters** applied to the tiles (**HSV colour/saturation** combined with **alpha-map**) to exaggerate the mediums-specific cues. This achieved an overall accuracy of **≈44%** on a balanced test set (1000 samples, 200 per class). This experiment proved the need to use a CNN to detect the features effectively.

Figure 14: *Heavy Combined Filter HSV Colour + Saturation Filter used for SVM*

Figure 15: *SVM Iteration Classification Report*

	precision	recall	f1-score	support
fresco_mural	0.35	0.33	0.34	200
gouache	0.57	0.74	0.65	200
oil	0.43	0.47	0.45	200
tempera	0.47	0.41	0.44	200
watercolour	0.32	0.25	0.28	200
accuracy			0.44	1000
macro avg	0.43	0.44	0.43	1000
weighted avg	0.43	0.44	0.43	1000

Iteration 2 – Various VGG16 Models

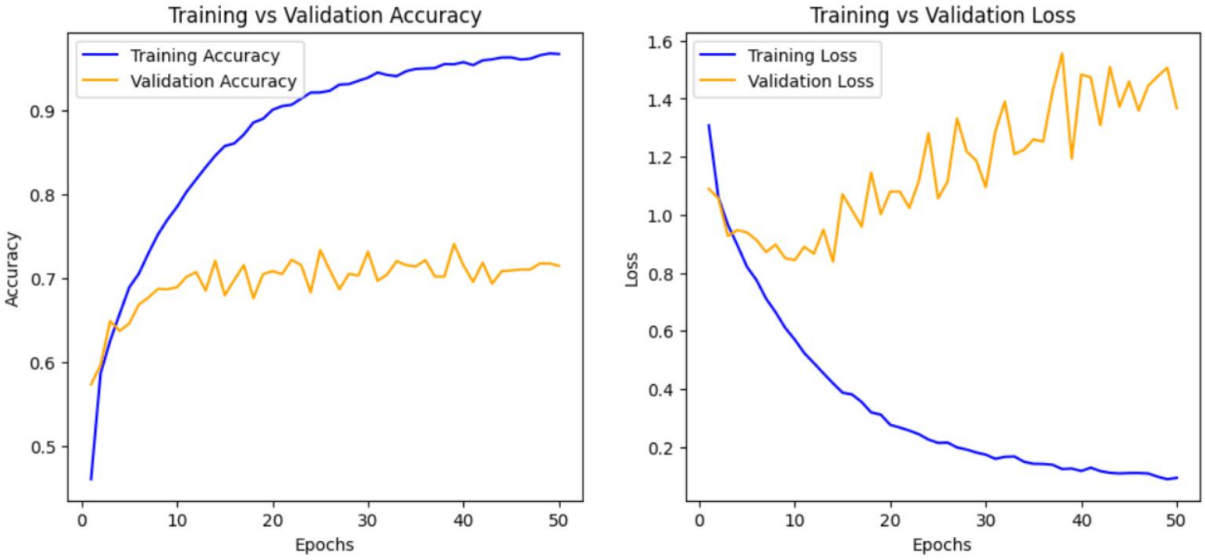
For the second range of model iterations, I focused on using VGG16, due to its effective ability in capturing hierarchical texture and edge patterns relevant for the fine-grained medium differences. This model was trained using a **frozen convolutional base**, a **custom head** (GlobalAveragePooling2D → Dense 256/512 ReLU → Dropout 0.3–0.5 → Dense 5 softmax), an **Adam optimizer** (lr ≈1e-4), **sparse categorical cross-entopy**, **data augmentation**, **early stopping** and **ReduceRLOnPlateau** in later runs to mitigate overfitting.

While the best performing VGG model was significantly better compared to the SVM baseline, the best accuracy achieved was **~76%** with the main issue being persistent **overfitting**.

Figure 16: *VGG16 Best Iteration Classification Report*

	precision	recall	f1-score	support
fresco_mural	0.65	0.87	0.74	500
gouache	0.84	0.77	0.80	500
oil	0.69	0.68	0.68	500
tempera	0.86	0.65	0.74	500
watercolour	0.80	0.82	0.81	500
accuracy			0.76	2500
macro avg	0.77	0.76	0.76	2500
weighted avg	0.77	0.76	0.76	2500

Figure 17: *VGG16 Best Iteration Overfitting Curves*



Iteration 3: Final Model Configuration (EfficientNetV2-S)

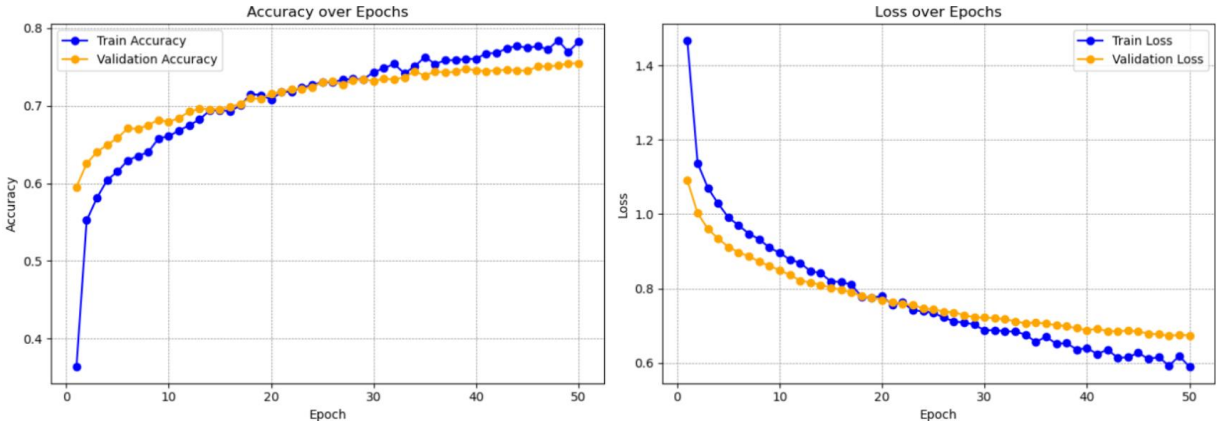
The final and best-performing model is a custom-trained EfficientNetV2-S CNN. This model was trained using a **fully frozen backbone** (pre-trained on ImageNet), with a **lightweight head** (GlobalAveragePooling2D, Dense 512 ReLu, Dropout 0.5 and Dense 5 softmax). The training was once again setup with an **Adam optimizer** (tuned lr ≈1e-4 to 5e-4), sparse categorical cross-entropy, data augmentation, **EarlyStopping** (patience 8-10) and **ReduceRLOnPlateau**.

This resulted in **not having overfitting**, which was the main problem using the VGG16 model, as well as stable training dynamics which led to efficient generalization and balanced performance across at 5 classes. The resulted in an overall per-tile accuracy yielding **~75.5%** and full-painting majority-vote accuracy reached **~93%** on 500 validation artworks.

Figure 18: *EfficientNetV2-S Best Iteration Classification Report*

	precision	recall	f1-score	support
fresco_mural	0.7301	0.7520	0.7409	500
gouache	0.7472	0.6740	0.7087	500
oil	0.7281	0.8300	0.7757	500
tempera	0.8574	0.8300	0.8435	500
watercolour	0.7167	0.6880	0.7020	500
accuracy			0.7548	2500
macro avg	0.7559	0.7548	0.7542	2500
weighted avg	0.7559	0.7548	0.7542	2500

Figure 19: *EfficientNetV2-S Best Iteration Overfitting Curves*

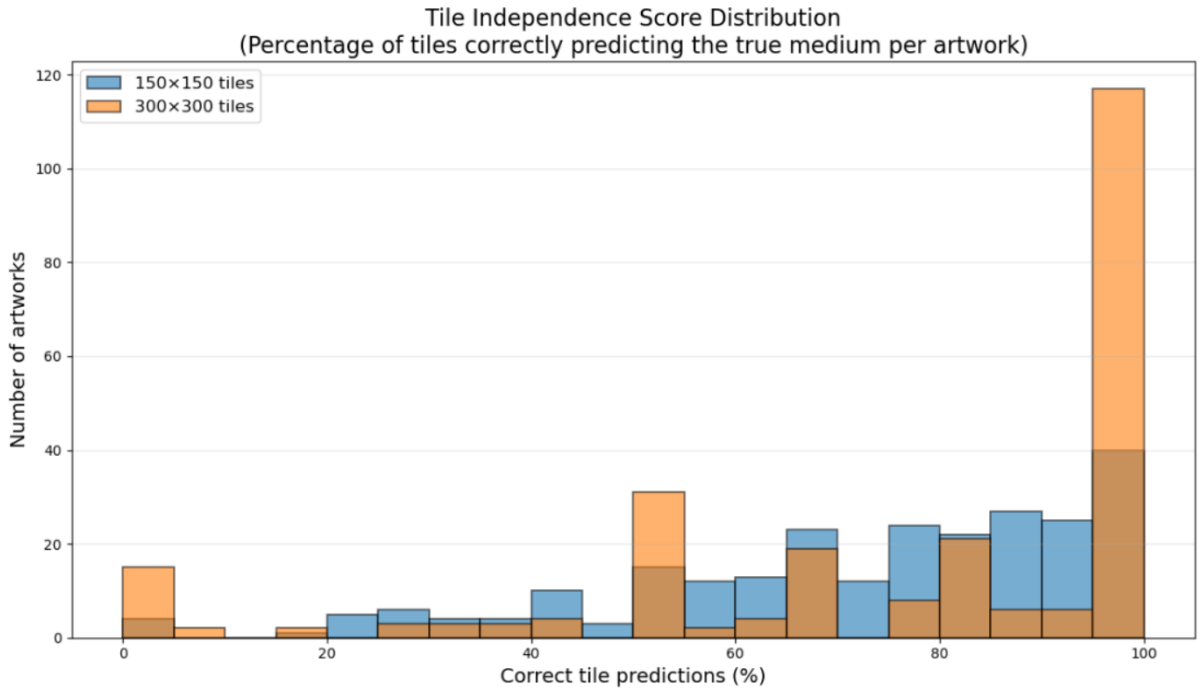


Optimal Tile Size

In all the above modelling sections, using the optimal tile size (150x150) played an important role in achieving such a high-level of accuracy. To determine the optimal tile size, I trained 2 instances of the final EfficientNetV2-S model, that were trained under identical conditions except for the tile size, trained on 150x150px and 300x300px respectively.

The performance was evaluated at both the tile-level (balanced test set accuracy) and via a **tile independence score**, which represents the per-tile distribution correctness with full artworks.

Figure 20: *Tile Independence Score Graph*



The results from this, combined with practical modelling and tested proved that the 150x150 tiles provide more robust predictions that have a lower variance, whereas the 300x300 tiles achieved high peak accuracy, however faced poor performance when the tiles included ambiguous regions.

Evaluation

This evaluation was based on the best performing final EfficientNetV2-S model.

Tile-level Performance

For the tile-level overall accuracy across all classes, it was tested on 500 tiles per class (2,500 total), where it scored **≈75.5%**. To view the entire classification report, please report to *Figure 18*.

The strongest performance was for tempera and oil; it was however much weaker for the water-based media (gouache and watercolour).

Full-Artwork Performance

To measure the performance for the application and real use-case, this process was tested using the end-to-end tile-based workflow (see section: *Tile-Based Workflow*), in brief terms, this consists of cropping the painting out of the frame, splitting tiles, filtering out noisy ones, per-tile prediction and majority vote across tiles. This process was tested on 100 validation images per class (500 in total). The overall accuracy is **92.8%**, which shows excellent performance on the realistic usage of the application.

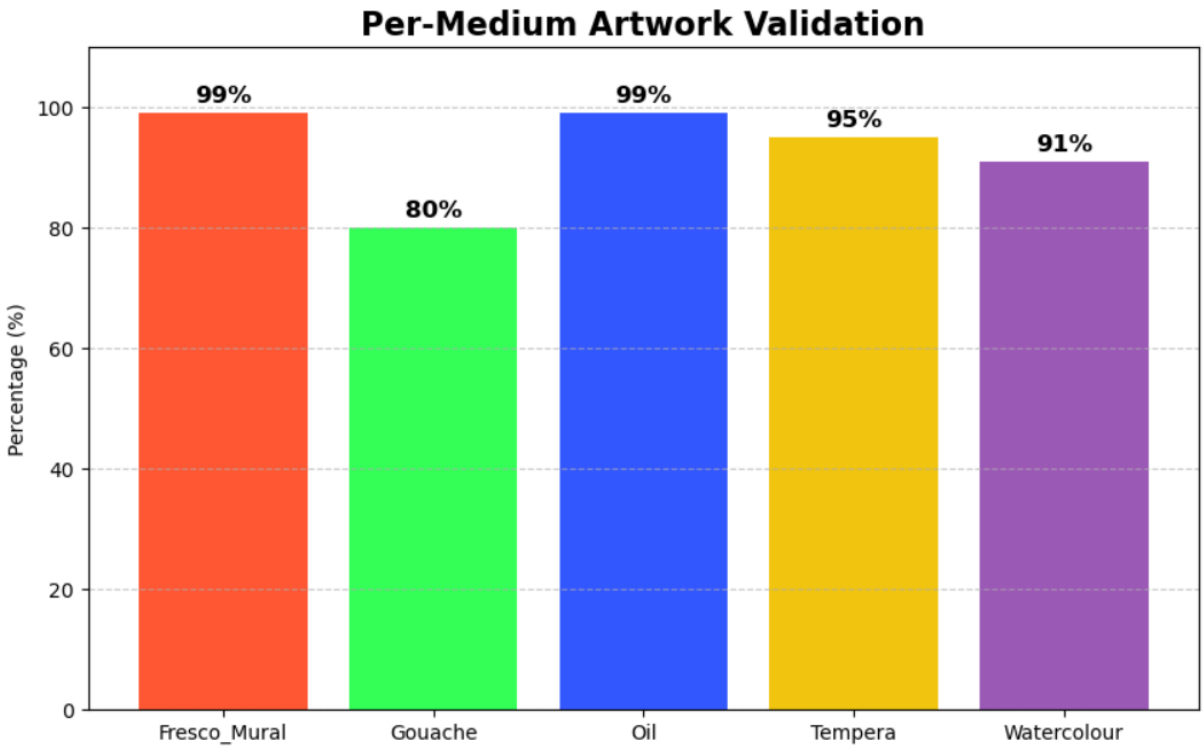
Figure 21: *Full-Artwork Validation Results Summary*

```
VALIDATION SUMMARY
============================================================
Total images processed: 500
Correct predictions:    464 (92.8%)
Incorrect predictions:  36 (7.2%)
Failed/Error:           0 (0.0%)

Accuracy (excluding failures): 92.80%
```

Regarding the per-medium accuracy, the model performed most strongly on fresco/mural and oil (99%), it also performed very well on tempera and watercolour (95% and 91% respectively), whereas it performed the lowest on gouache (80%), which is due to the initial dataset imbalance and its close resemblance to watercolour.

Figure 22: *Per-Medium Artwork Validation Report*



Confusion Patterns

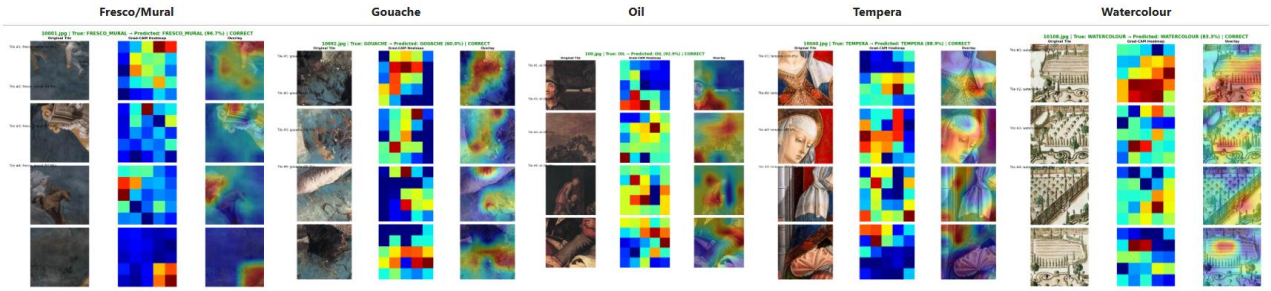
The main confusion faced by the model from full-artwork validation (and tile-level classification report) was between the water-based media (gouache and watercolour), the transparency is similar, making it difficult to train the model to tell them apart efficiently. There is some moderate confusion between oil and tempera as they are both opaque.

Error Analysis and Model Interpretability (Grad-CAM)

Grad-CAM was used for the error analysis; this points out the regions where the model focuses on. This was applied to the final EfficientNetV2-S model on the validation tiles.

My key finding was that for most classes the model looks across the tile, in a broad, distributed way. For fresco/mural, it often highlights cracks, plaster or rough surface patterns, which is correct. For oil, it sometimes highlights impasto ridges, colour blending or a weaving canvas. For tempera, it focuses on fine details like cross-hatching. Finally, for watercolour and gouache the attention is more spread out and focused on soft gradients and light paint over a paper texture. All of this aligns with the domain knowledge (see section: *Domain Study*).

Figure 23: *Grad-CAM Validation Tile Examples*



Discussion

Key Findings and Lessons Learned

The data cleaning, cropping, multi-stage filtering and minor class complementation proved more impactful than model architecture changes individually. Heavy filters to differentiate the features of different paintings proved to be counterproductive and had a negative impact on the SVM, confirming that model CNN's learn best directly from raw RGB tiles.

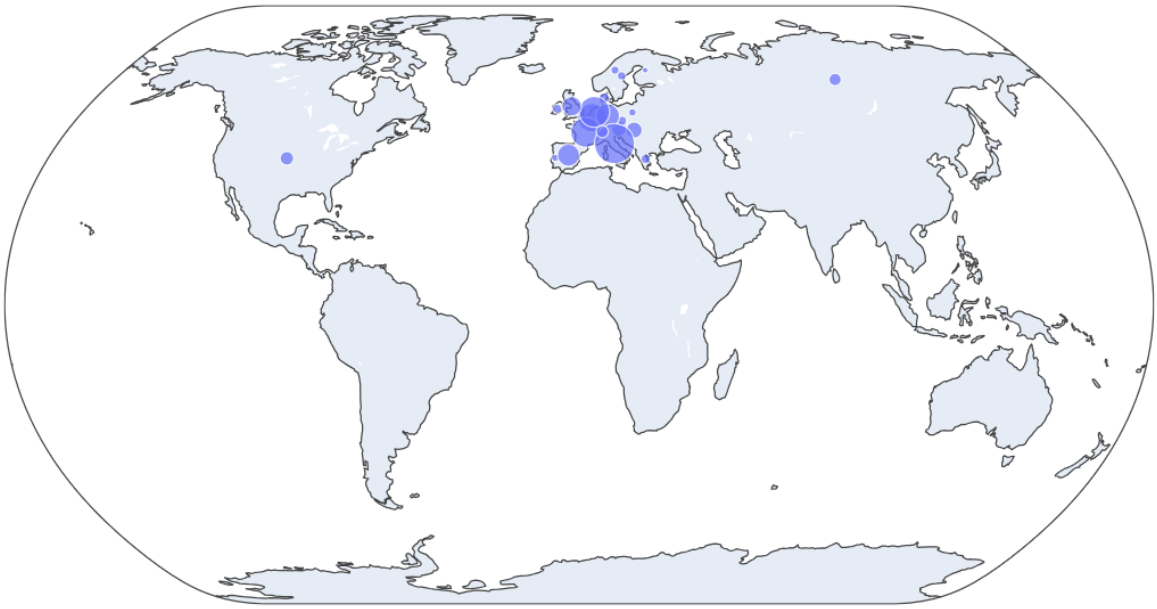
The consistent alignment between confusion patterns, Grad-CAM visualizations, and established art-domain knowledge further validates that the model captures genuine medium-specific cues rather than dataset artifacts.

Limitations

The limitations for the current approach are the following:

1. **Gouache performance:** This remains to be the weakest class (~80% full-painting accuracy) due to its very small original sample size and a reliance on a supplementary dataset. Additionally, this is structurally very similar to watercolour, making it difficult to distinguish them.
2. **Dataset Bias and Scope:** The dataset is strongly biased towards historical Western European art; therefore, it may not generalize well towards model or contemporary paintings.

Figure 24: *Geographical Origin of Paintings in the ART500K Toy Dataset*



3. **Real-World Robustness:** The model is trained on clean, high-resolution, frontal and well-lit images, this performance could significantly degrade when using smartphone pictures
4. **No Rejection Mechanism:** The system always provides a prediction, even for low-quality of art pieces it cannot handle (like statues).

Conclusion and Presentation

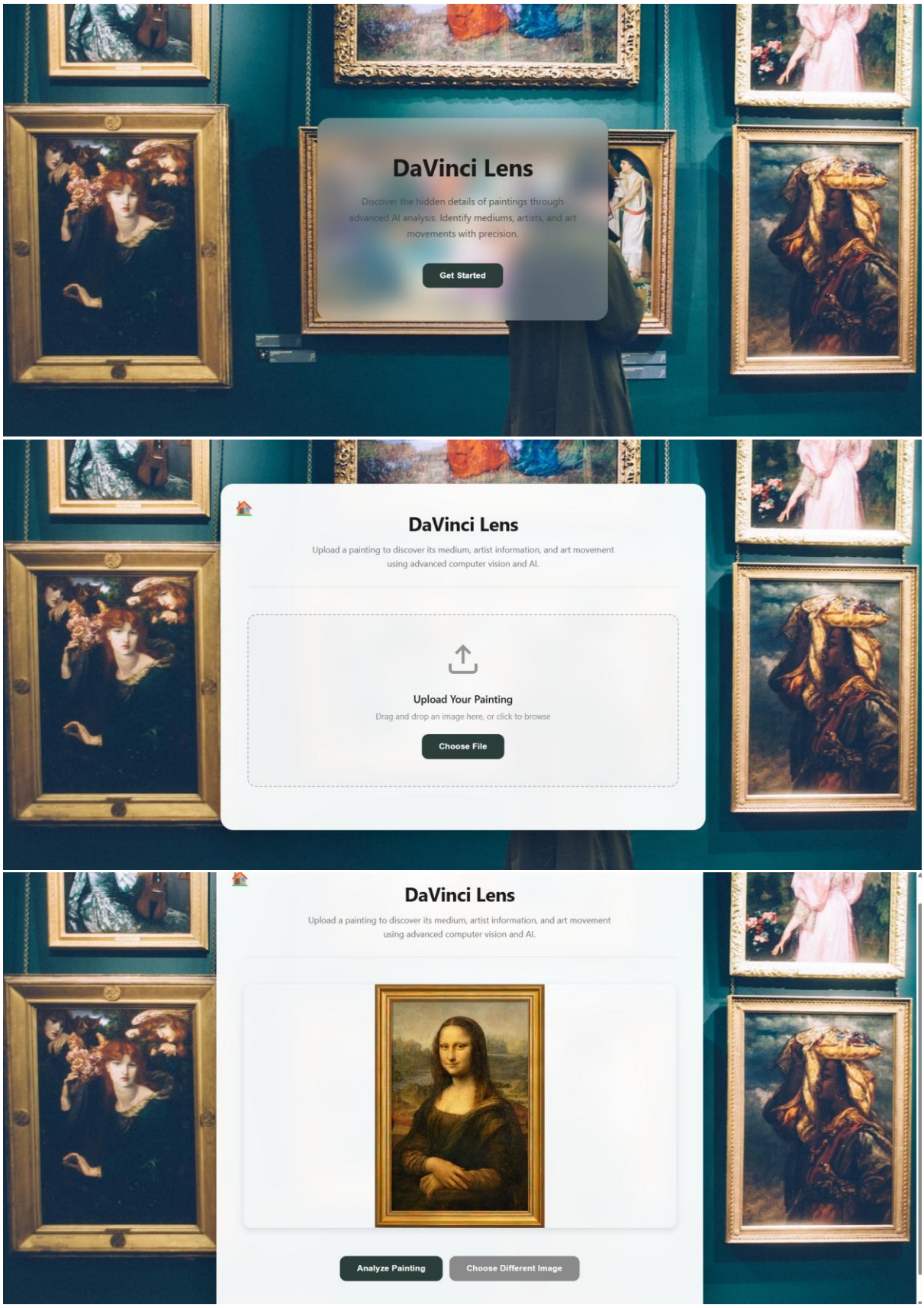
Summary of Achievements

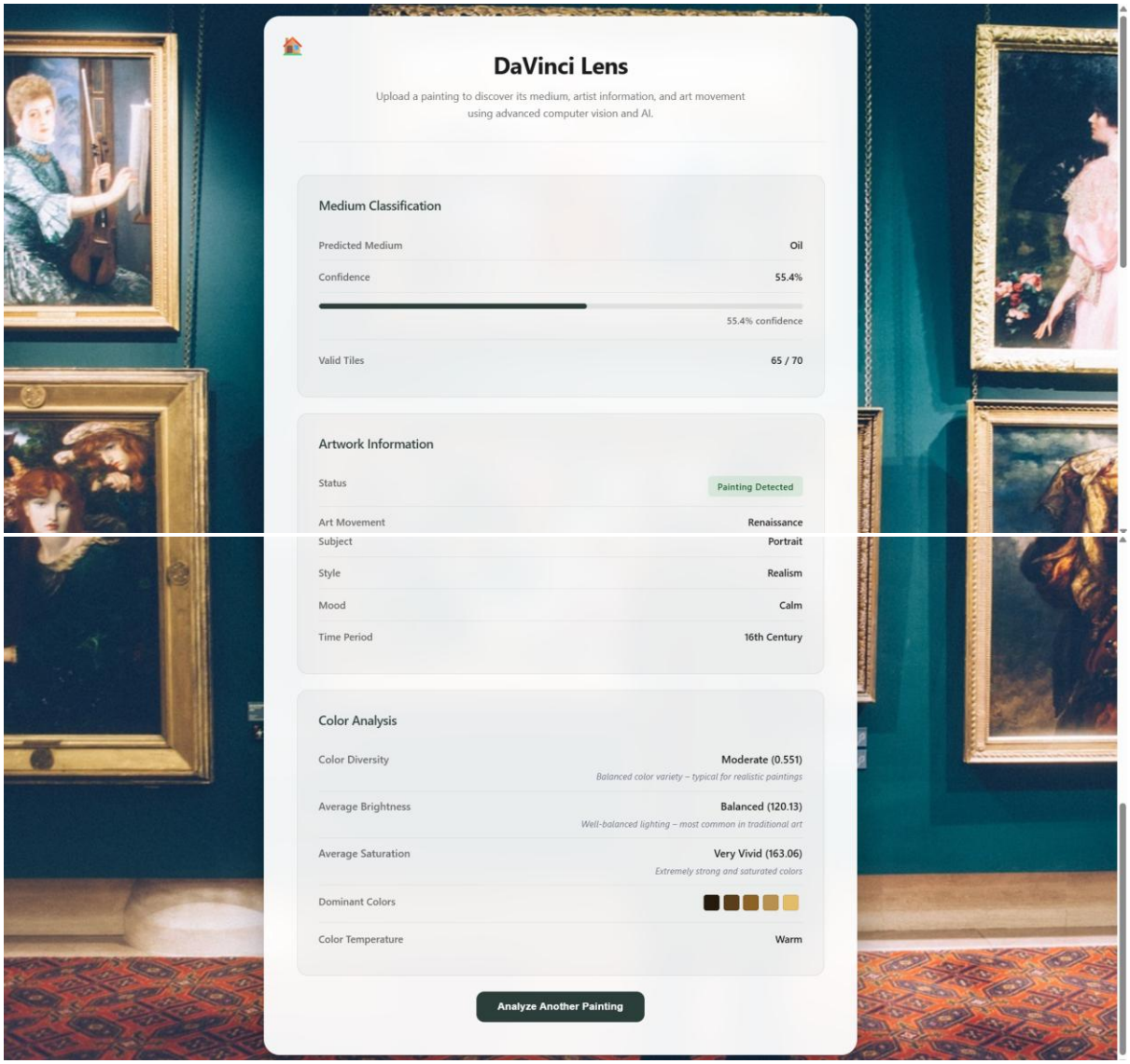
The university project was a success in building DaVinci Lens, the high accuracy per-tile and especially for full artworks was achieved through extensive data cleaning, tile-based processing, multi-stage filtering as well as majority-vote inference. The final model yielded ~75% balanced tile-level accuracy and ~93% full-artwork accuracy.

Web Application

A web application was built to demonstrate the project, this consists of a **REST API** for the backend built using **FastAPI**, the frontend was developed using **HTML**, **CSS** and **Vanilla JavaScript**. The web application enables the user to upload a painting, information about the painting medium, as well as some information about the art movement, subject and era using a **VLM** and useful colour statistics.

Figure 25: *DaVinci Lens Web Application Screenshots*




DaVinci Lens
Upload a painting to discover its medium, artist information, and art movement using advanced computer vision and AI.
Medium Classification
Predicted Medium
Oil
Confidence
55.4%
55.4% confidence
Valid Tiles
65 / 70
Artwork Information
Status
Painting Detected
Art Movement
Renaissance
Subject
Portrait
Style
Realism
Mood
Calm
Time Period
16th Century
Color Analysis
Color Diversity
Moderate (0.551)
Balanced color variety – typical for realistic paintings
Average Brightness
Balanced (120.13)
Well-balanced lighting – most common in traditional art
Average Saturation
Very Vivid (163.06)
Extremely strong and saturated colors
Dominant Colors
Color Temperature
Warm
Analyze Another Painting

References

Hong Kong University of Science and Technology. (n.d.). *ART500K dataset*. DeepArt at HKUST.
https://deepart.hkust.edu.hk/ART500K/art500k.html

Cleveland Museum of Art. (n.d.). *Open access API*. The Cleveland Museum of Art.
https://www.clevelandart.org/open-access-api

Kingma, D. P., & Ba, J. (2015). *Adam: A method for stochastic optimization*. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*.
https://arxiv.org/abs/1412.6980

Loshchilov, I., & Hutter, F. (2019). *Decoupled weight decay regularization*. In *International Conference on Learning Representations (ICLR)*.
https://arxiv.org/abs/1711.05101

Mingxing, T., & Le, Q. V. (2021). *EfficientNetV2: Smaller models and faster training*. *Proceedings of the 38th International Conference on Machine Learning (ICML)*.
https://arxiv.org/abs/2104.00298

Simonyan, K., & Zisserman, A. (2015). *Very deep convolutional networks for large-scale image recognition*. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*.
https://arxiv.org/abs/1409.1556

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). *Grad-CAM: Visual explanations from deep networks via gradient-based localization*. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)* (pp. 618–626). IEEE.
https://arxiv.org/abs/1610.02391

Cortes, C., & Vapnik, V. (1995). *Support-vector networks*. *Machine Learning*, *20*(3), 273–297.
https://doi.org/10.1007/BF00994018

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019). *PyTorch: An imperative style, high-performance deep learning library*. In *Advances in Neural Information Processing Systems (NeurIPS 2019)*.
https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library

Chollet, F. (2015). *Keras*. GitHub repository.
https://keras.io

van Rossum, G., & Drake, F. L. (2009). *Python 3 reference manual*. CreateSpace.
https://www.python.org

Grinberg, M. (2023). *FastAPI framework documentation*. *FastAPI*.
https://fastapi.tiangolo.com

Mozilla Developer Network (MDN). (2024). *HTML: HyperText Markup Language*. *MDN Web Docs.*
https://developer.mozilla.org/en-US/docs/Web/HTML

Mozilla Developer Network (MDN). (2024). *CSS: Cascading Style Sheets*. *MDN Web Docs.*
https://developer.mozilla.org/en-US/docs/Web/CSS

Mozilla Developer Network (MDN). (2024). *JavaScript guide*. *MDN Web Docs.*
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

Google Developers. (2024). *TensorFlow: An end-to-end machine learning platform*. *TensorFlow.*
https://www.tensorflow.org

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). *Dropout: A simple way to prevent neural networks from overfitting*. *Journal of Machine Learning Research*, *15*(1), 1929–1958.
https://jmlr.org/papers/v15/srivastava14a.html

Turing, A. M. (1950). *Computing machinery and intelligence*. *Mind*, *59*(236), 433–460.
https://doi.org/10.1093/mind/LIX.236.433